

Smart Digital Lock System Using VHDL

FPGA-Based Security System with FSM Control

Project Title

Smart Digital Lock System Using VHDL

Abstract

The Smart Digital Lock System is an FPGA-based security application developed using VHDL. The system authenticates users through a password entered using a keypad interface. The entered password is verified against a predefined password stored within the FPGA logic. If the password is correct, the lock is opened; otherwise, the system counts invalid attempts and activates an alarm after multiple failed entries.

The project demonstrates practical implementation of Finite State Machine (FSM) concepts, sequential logic, keypad interfacing, display control, and FPGA deployment.

Objectives

- Design a digital lock system using VHDL.
 - Implement password authentication using FSM.
 - Interface keypad and 7-segment display.
 - Generate alerts for invalid attempts.
 - Simulate and synthesize the design using FPGA tools.
 - Deploy the design on FPGA hardware.
-

Features

- 4-digit password authentication
- FSM-based control logic
- Wrong password detection
- Alarm after 3 failed attempts
- Master reset functionality
- 7-segment display output
- LED indication for lock/unlock status

- FPGA implementation support

Software Requirements

Software	Purpose
Vivado	FPGA synthesis and implementation
ModelSim	Simulation and waveform analysis
VHDL	Hardware Description Language

Hardware Requirements

Hardware	Purpose
FPGA Board (Spartan/Artix)	Main platform
4x4 Keypad	Password input
7-Segment Display	Display output
LEDs	Status indication
Push Button	Reset
Buzzer (Optional)	Alarm indication

System Architecture

```
+-----+
| Keypad Interface |
+-----+-----+
      |
      v
+-----+-----+
| Password Register |
+-----+-----+
      |
      v
+-----+-----+
| Password Compare  |
+-----+-----+
```

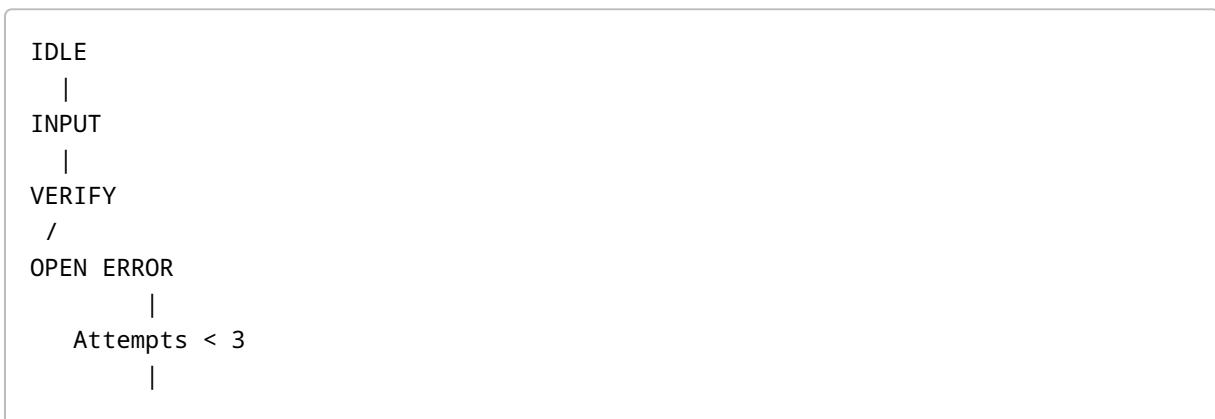


Finite State Machine (FSM)

States

State	Description
IDLE	Waiting for password
INPUT	Accepting password digits
VERIFY	Password verification
OPEN	Correct password
ERROR	Wrong password
ALARM	3 wrong attempts
RESET	Reset operation

FSM Flow Diagram





Working Principle

Step 1: System Initialization

- System enters IDLE state.
- Display shows LOCK.

Step 2: Password Entry

- User enters 4-digit password using keypad.
- Digits are stored sequentially.

Step 3: Verification

- Entered password is compared with stored password.

If Correct:

- Lock opens.
- Green LED turns ON.
- Display shows OPEN.

If Incorrect:

- Red LED turns ON.
- Attempt counter increments.

Step 4: Alarm Condition

- After 3 incorrect attempts:
- Alarm activates.
- Input is blocked.

Step 5: Reset

- Reset button clears alarm and attempt counter.

- FSM returns to IDLE state.

VHDL Code

Main FSM Module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity smart_lock is
    Port (
        clk      : in STD_LOGIC;
        reset    : in STD_LOGIC;
        password  : in STD_LOGIC_VECTOR(15 downto 0);
        unlock   : out STD_LOGIC;
        alarm    : out STD_LOGIC
    );
end smart_lock;

architecture Behavioral of smart_lock is

    type state_type is (
        IDLE,
        VERIFY,
        OPEN,
        ERROR,
        ALARM_STATE
    );

    signal current_state, next_state : state_type;

    constant correct_password : STD_LOGIC_VECTOR(15 downto 0)
        := "0001001000110100";

    signal attempts : integer range 0 to 3 := 0;

begin

    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= IDLE;
            attempts <= 0;
```

```

        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

process(current_state, password, attempts)
begin

    unlock <= '0';
    alarm <= '0';

    case current_state is

        when IDLE =>
            next_state <= VERIFY;

        when VERIFY =>
            if password = correct_password then
                next_state <= OPEN;
            else
                next_state <= ERROR;
            end if;

        when OPEN =>
            unlock <= '1';
            next_state <= IDLE;

        when ERROR =>
            if attempts < 2 then
                next_state <= IDLE;
            else
                next_state <= ALARM_STATE;
            end if;

        when ALARM_STATE =>
            alarm <= '1';
            next_state <= ALARM_STATE;

        when others =>
            next_state <= IDLE;

    end case;

end process;

process(clk)
begin

```

```

    if rising_edge(clk) then
        if current_state = ERROR then
            attempts <= attempts + 1;
        elsif current_state = OPEN then
            attempts <= 0;
        end if;
    end if;
end process;

end Behavioral;

```

Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_smart_lock is
end tb_smart_lock;

architecture Behavioral of tb_smart_lock is

    signal clk      : STD_LOGIC := '0';
    signal reset    : STD_LOGIC := '0';
    signal password  : STD_LOGIC_VECTOR(15 downto 0);
    signal unlock   : STD_LOGIC;
    signal alarm    : STD_LOGIC;

    component smart_lock
    Port (
        clk      : in STD_LOGIC;
        reset    : in STD_LOGIC;
        password  : in STD_LOGIC_VECTOR(15 downto 0);
        unlock   : out STD_LOGIC;
        alarm    : out STD_LOGIC
    );
end component;

begin

    uut: smart_lock
    port map (
        clk => clk,
        reset => reset,
        password => password,

```

```

        unlock => unlock,
        alarm => alarm
    );

    clk_process : process
    begin
        clk <= '0';
        wait for 5 ns;
        clk <= '1';
        wait for 5 ns;
    end process;

    stimulus : process
    begin

        reset <= '1';
        wait for 20 ns;
        reset <= '0';

        password <= "0001001000110100";
        wait for 50 ns;

        password <= "1111111111111111";
        wait for 50 ns;

        wait;

    end process;

end Behavioral;

```

Simulation Results

Expected Outputs

Correct Password

Input	Output
1234	OPEN

Wrong Password

Input	Output
9999	ERROR

Alarm Condition

Attempts	Output
3 Wrong Entries	ALARM ON

FPGA Implementation Steps

1. Open Vivado.
2. Create new VHDL project.
3. Add source files.
4. Add constraints (.xdc) file.
5. Run synthesis.
6. Run implementation.
7. Generate bitstream.
8. Program FPGA board.

Pin Configuration Example

FPGA Pin	Device
P1-P4	Keypad Rows
P5-P8	Keypad Columns
P9	Green LED
P10	Red LED
P11	Alarm
P12-P18	7-Segment Display

Advantages

- Secure digital authentication

- Real-time operation
 - Reprogrammable FPGA logic
 - Compact design
 - Low hardware cost
-

Applications

- Smart door locks
 - Office security systems
 - ATM authentication systems
 - Industrial access control
 - Home automation security
-

Future Enhancements

- RFID integration
 - Fingerprint authentication
 - Bluetooth control
 - GSM-based security alerts
 - EEPROM password storage
 - Mobile app integration
-

Results

The Smart Digital Lock System was successfully designed and simulated using VHDL. The FSM correctly handled password authentication, unlock operation, error detection, and alarm activation. The design is suitable for FPGA implementation and demonstrates practical digital system design concepts.

Conclusion

This project demonstrates the implementation of a secure FPGA-based digital lock system using VHDL. It provides hands-on experience in FSM design, hardware description languages, digital security systems, simulation, and FPGA deployment. The project is highly suitable for academic mini-projects, internships, and beginner-level VLSI/FPGA applications.

GitHub Repository Structure

```
Smart-Digital-Lock-System/  
|  
├── VHDL_Code/  
|   ├── smart_lock.vhd  
|   └── tb_smart_lock.vhd  
|  
├── Simulation/  
|   └── waveform_screenshots  
|  
├── FPGA_Implementation/  
|   └── constraints.xdc  
|  
├── Report/  
|   └── project_report.pdf  
|  
├── Images/  
|   └── block_diagram.png  
|  
└── README.md
```

Author

Name: Your Name

Internship Domain: VLSI / FPGA Design

Technology Used: VHDL, Vivado, ModelSim, FPGA

License

This project is for educational and internship purposes.